



Documentation for Graduate Admission Prediction Model



Overview

This project predicts whether a student will be admitted into a graduate program based on academic and personal features. It was submitted to Zindi's "From Scores to Seats: The Grad School ML Challenge".

The notebook contains:

- Data cleaning and preprocessing
 - Feature engineering
 - Class imbalance handling
 - Model training and validation
 - Submission file creation
-



Dataset Description

The dataset contains the following columns:

- **ID**: Student ID (not used for prediction)
- **GRE Score**: GRE test score
- **TOEFL Score**: TOEFL English proficiency test score
- **University Rating**: University rating
- **SOP**: Statement of Purpose strength (0–5)

- **LOR**: Letter of Recommendation strength (0–5)
 - **CGPA**: Cumulative GPA (0–10 scale)
 - **Research**: Research background (0 = No, 1 = Yes)
 - **Admitted**: Target column
 - **Age**: Applicant's age
 - **First_Generation**: First-gen college student (0 = No, 1 = Yes)
 - **Location_***: One-hot encoded location information
-

Approach

1. Data Preprocessing

- Removed CGPA anomalies (values > 10)
- Checked for and filled missing values
- Normalized numerical features with Standard Scaler
- Removed anomalies in the CGPA column

2. Feature Engineering

- Created new features:
 - **Research * LOR** to capture interaction
 - **GRE_TOEFL_Ratio** to capture student balance
 - **SOP_LOR_Avg** to capture average strength of the letter of recommendation and statement of purpose
- Addressed imbalance in the **Admitted** column using SMOTE
- One-hot encoded categorical location variables

3. Model Selection and Training

A wide variety of base learners and ensemble models were evaluated to identify the best-performing approach. The following models were used:

✓ Base Models:

- **Logistic Regression**
- **RidgeCV**
- **LassoCV**
- **Support Vector Classifier (SVC)**
- **LightGBMClassifier**
- **RandomForestClassifier**
- **ExtraTreesClassifier**
- **GradientBoostingClassifier**
- **AdaBoostClassifier**
- **DecisionTreesClassifier**

✓ Ensemble Techniques:

- **BaggingClassifier** with **LightGBM** as the base estimator
- **StackingClassifier** — Final ensemble combining multiple base models above (first 9)

Each model was evaluated using cross-validation to ensure generalizability and to reduce overfitting. The final predictions were generated using the **StackingClassifier**, which outperformed individual base models on the leaderboard.

Note: The stacking classifier also included **RidgeCV** and **LassoCV** as base learners.

While these models are primarily designed for regression tasks, they were included to capture linear relationships within the data and leverage their automatic hyperparameter tuning via

cross-validation. This choice ensured model diversity within the ensemble and contributed to its ability to generalize well across unseen data.

4. Evaluation

- Evaluated on confusion matrix, precision, recall, F1-score

5. Prediction & Submission

- Made predictions on the test dataset
- Created `submission.csv` for leaderboard upload

Requirements

A `requirements.txt` file was provided with all dependencies.

How to Run

1. Install the libraries:

```
pip install -r requirements.txt
```

2. Run the Jupyter Notebook (Google Colab): `Itopa_starternotebook.ipynb`
3. This generates a file: `submission.csv`

Author

Itopa
April 2025